

```
# Import libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Machine Learning libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Evaluation Metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve
)
```

```
# Import pandas
import pandas as pd

# Load the dataset
df = pd.read_csv("german_credit_data.csv")

# Display the first 5 rows
df.head()
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_1755/1020495792.py in <cell line: 0>()
      3
      4 # Load the dataset
----> 5 df = pd.read_csv("german_credit_data.csv")
      6
      7 # Display the first 5 rows
```

```
-----
4 frames
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression,
memory_map, is_text, errors, storage_options)
    871     if ioargs.encoding and "b" not in ioargs.mode:
    872         # Encoding
--> 873         handle = open(
    874             handle,
    875             ioargs.mode,
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'german_credit_data.csv'
```

Next steps: [Explain error](#)

```
import zipfile

# Extract the ZIP file
with zipfile.ZipFile('/content/archive (1).zip', 'r') as zip_ref:
    zip_ref.extractall('/content/')
```

```
import os

os.listdir('/content')

['.config', 'german_credit_data.csv', 'archive (1).zip', 'sample_data']
```

```
import pandas as pd

# Load the dataset
```

```
df = pd.read_csv('/content/german_credit_data.csv')

# Display first 5 rows
df.head()
```

|   | Unnamed: 0 | Age | Sex    | Job | Housing | Saving accounts | Checking account | Credit amount | Duration | Purpose             |
|---|------------|-----|--------|-----|---------|-----------------|------------------|---------------|----------|---------------------|
| 0 | 0          | 67  | male   | 2   | own     | NaN             | little           | 1169          | 6        | radio/TV            |
| 1 | 1          | 22  | female | 2   | own     | little          | moderate         | 5951          | 48       | radio/TV            |
| 2 | 2          | 49  | male   | 1   | own     | little          | NaN              | 2096          | 12       | education           |
| 3 | 3          | 45  | male   | 2   | free    | little          | little           | 7882          | 42       | furniture/equipment |
| 4 | 4          | 53  | male   | 2   | free    | little          | little           | 4870          | 24       | car                 |

Next steps:

Generate code with df

New interactive sheet

```
# Shape of dataset
print("Shape:", df.shape)

# Column names
print("\nColumns:")
print(df.columns)

# Dataset information
print("\nDataset Information:")
df.info()
```

Shape: (1000, 10)

Columns:

Index(['Unnamed: 0', 'Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account', 'Credit amount', 'Duration', 'Purpose'], dtype='object')

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            1000 non-null  int64
1   Age                   1000 non-null  int64
2   Sex                   1000 non-null  object
3   Job                   1000 non-null  int64
4   Housing               1000 non-null  object
5   Saving accounts       817 non-null   object
6   Checking account      606 non-null   object
7   Credit amount         1000 non-null  int64
8   Duration              1000 non-null  int64
9   Purpose               1000 non-null  object
dtypes: int64(5), object(5)
memory usage: 78.3+ KB
```

```
# Check missing values
print(df.isnull().sum())
```

```
Unnamed: 0      0
Age             0
Sex             0
Job             0
Housing         0
Saving accounts 183
Checking account 394
Credit amount   0
Duration        0
Purpose         0
dtype: int64
```

```
# Remove unnecessary column
df.drop("Unnamed: 0", axis=1, inplace=True)

# Fill missing values
df["Saving accounts"].fillna("Unknown", inplace=True)
df["Checking account"].fillna("Unknown", inplace=True)

# Check missing values again
```

```
# Check missing values again
df.isnull().sum()
```

/tmp/ipykernel\_1755/3110688671.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through cha  
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are se

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] =

```
df["Saving accounts"].fillna("Unknown", inplace=True)
```

/tmp/ipykernel\_1755/3110688671.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through cha  
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are se

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] =

```
df["Checking account"].fillna("Unknown", inplace=True)
```

|                  | 0 |
|------------------|---|
| Age              | 0 |
| Sex              | 0 |
| Job              | 0 |
| Housing          | 0 |
| Saving accounts  | 0 |
| Checking account | 0 |
| Credit amount    | 0 |
| Duration         | 0 |
| Purpose          | 0 |

dtype: int64

df.head()

|   | Age | Sex    | Job | Housing | Saving accounts | Checking account | Credit amount | Duration | Purpose             |  |
|---|-----|--------|-----|---------|-----------------|------------------|---------------|----------|---------------------|--|
| 0 | 67  | male   | 2   | own     | Unknown         | little           | 1169          | 6        | radio/TV            |  |
| 1 | 22  | female | 2   | own     | little          | moderate         | 5951          | 48       | radio/TV            |  |
| 2 | 49  | male   | 1   | own     | little          | Unknown          | 2096          | 12       | education           |  |
| 3 | 45  | male   | 2   | free    | little          | little           | 7882          | 42       | furniture/equipment |  |
| 4 | 53  | male   | 2   | free    | little          | little           | 4870          | 24       | car                 |  |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
for col in df.columns:
    print(f"\n{col}")
    print(df[col].unique())
```

```

3349 2828 4526 2671 2051 1300 741 3357 3632 1808 12204 9157
3676 3441 640 3652 1530 3914 1858 2600 1979 2116 1437 4042
3660 1444 1980 1355 1376 15653 1493 4370 750 1308 4623 1851
1880 7980 4583 1386 947 684 7476 1922 2303 8086 2346 3973
888 10222 4221 6361 1297 900 1050 1047 6314 3496 3609 4843
4139 5742 10366 2080 2580 4530 5150 5595 1453 1538 2279 5103
9857 6527 1347 2862 2753 3651 975 2896 4716 2284 1103 926
1800 1905 1123 6331 1377 2503 2528 5324 6560 2969 1206 2118
629 1198 2476 1138 14027 7596 1505 3148 6148 1337 1228 790
2570 250 1316 1882 6416 6403 1987 760 2603 3380 3990 11560
4380 6761 4280 2325 1048 3160 2483 14179 1797 2511 1274 5248
3029 428 976 841 5771 1555 1285 1299 1271 691 5045 2124
2214 12680 2463 1155 3108 2901 1655 2812 8065 3275 2223 1480
1371 3535 3509 5711 3872 4933 1940 836 1941 2675 2751 6224
5998 1188 6313 1221 2892 3062 2301 7511 1549 1795 7472 9271
590 930 9283 1778 907 484 9629 3051 3931 7432 1338 1554
15857 1345 1101 3016 2712 731 3780 1602 3966 4165 8335 6681
2375 11816 5084 2327 886 601 2957 2611 5179 2993 1943 1559
3422 3976 1249 2235 1471 10875 894 3343 3959 3577 5804 2169
2439 2210 2221 2389 3331 7409 652 7678 1343 874 3590 1322
3595 1422 6742 7814 9277 2181 1098 4057 795 2825 15672 6614
7824 2442 1829 5800 8947 2606 1592 2186 4153 2625 3485 10477
1278 1107 3763 3711 3594 3195 4454 4736 2991 2142 3161 18424
2848 14896 2359 3345 1817 12749 1366 2002 6872 697 1049 10297
1867 1747 1670 1224 522 1498 745 2063 6288 6842 3527 929
1455 1845 8358 2859 3621 2145 4113 10974 1893 3656 4006 3069
1740 2353 3556 2397 454 1715 2520 3568 7166 3939 1514 7393
1193 7297 2831 753 2427 2538 8386 4844 2923 8229 1433 6289
6579 3565 1569 1936 2390 1736 3857 804 4576]

```

Duration

```
[ 6 48 12 42 24 36 30 15 9 10 7 60 18 45 11 27 8 54 20 14 33 21 16 4
47 13 22 39 28 5 26 72 40]
```

Purpose

```
['radio/TV' 'education' 'furniture/equipment' 'car' 'business']
```

df.columns

```
Index(['Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account',
      'Credit amount', 'Duration', 'Purpose'],
      dtype='object')
```

df.columns

```
Index(['Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account',
      'Credit amount', 'Duration', 'Purpose'],
      dtype='object')
```

df.columns

```
Index(['Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account',
      'Credit amount', 'Duration', 'Purpose'],
      dtype='object')
```

df.columns

```
Index(['Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account',
      'Credit amount', 'Duration', 'Purpose'],
      dtype='object')
```

df.columns

```
Index(['Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account',
      'Credit amount', 'Duration', 'Purpose'],
      dtype='object')
```

```
import pandas as pd
```

```
df = pd.read_csv("/content/german_credit_data_updated.csv")
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_1755/2710521690.py in <cell line: 0>()
      1 import pandas as pd
      2
----> 3 df = pd.read_csv("/content/german_credit_data_updated.csv")

4 frames
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression,
memory_map, is_text, errors, storage_options)
      871     if ioargs.encoding and "b" not in ioargs.mode:
      872         # Encoding
--> 873         handle = open(
      874             handle,
      875             ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: '/content/german_credit_data_updated.csv'
```

Next steps: [Explain error](#)

```
import os

print(os.listdir('/content'))

['.config', '.ipynb_checkpoints', 'archive (3).zip', 'sample_data']
```

```
import zipfile

with zipfile.ZipFile('/content/archive (3).zip', 'r') as zip_ref:
    zip_ref.extractall('/content')
```

```
import os
print(os.listdir('/content'))

['.config', 'reintegrate_credit_score.py', '.ipynb_checkpoints', 'archive (3).zip', 'german_credit_data_updated.csv', 'sample_
```

```
import os
print(os.listdir('/content'))

['.config', 'reintegrate_credit_score.py', '.ipynb_checkpoints', 'archive (3).zip', 'german_credit_data_updated.csv', 'sample_
```

```
import pandas as pd

df = pd.read_csv("/content/german_credit_data_updated.csv")
```

df.head()

|   | Unnamed: 0 | Age | Sex    | Job | Housing | Saving accounts | Checking account | Credit amount | Duration | Purpose             | Credit Risk |
|---|------------|-----|--------|-----|---------|-----------------|------------------|---------------|----------|---------------------|-------------|
| 0 | 0          | 67  | male   | 2   | own     | NaN             | little           | 1169          | 6        | radio/TV            | 1           |
| 1 | 1          | 22  | female | 2   | own     | little          | moderate         | 5951          | 48       | radio/TV            | 2           |
| 2 | 2          | 49  | male   | 1   | own     | little          | NaN              | 2096          | 12       | education           | 1           |
| 3 | 3          | 45  | male   | 2   | free    | little          | little           | 7882          | 42       | furniture/equipment | 1           |
| 4 | 4          | 53  | male   | 2   | free    | little          | little           | 4870          | 24       | car                 | 2           |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.columns

Index(['Unnamed: 0', 'Age', 'Sex', 'Job', 'Housing', 'Saving accounts',
      'Checking account', 'Credit amount', 'Duration', 'Purpose',
      'Credit Risk'],
      dtype='object')
```

```
# Remove the unnecessary index column
df.drop("Unnamed: 0", axis=1, inplace=True)
```

```
# Fill missing values
df["Saving accounts"] = df["Saving accounts"].fillna("Unknown")
df["Checking account"] = df["Checking account"].fillna("Unknown")

# Verify there are no missing values
print(df.isnull().sum())
```

```
Age          0
Sex          0
Job          0
Housing      0
Saving accounts 0
Checking account 0
Credit amount 0
Duration     0
Purpose      0
Credit Risk  0
dtype: int64
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_columns = [
    "Sex",
    "Housing",
    "Saving accounts",
    "Checking account",
    "Purpose",
    "Credit Risk"
]

for col in categorical_columns:
    df[col] = le.fit_transform(df[col])

# Display the first 5 rows after encoding
df.head()
```

|   | Age | Sex | Job | Housing | Saving accounts | Checking account | Credit amount | Duration | Purpose | Credit Risk |
|---|-----|-----|-----|---------|-----------------|------------------|---------------|----------|---------|-------------|
| 0 | 67  | 1   | 2   | 1       | 0               | 1                | 1169          | 6        | 5       | 0           |
| 1 | 22  | 0   | 2   | 1       | 1               | 2                | 5951          | 48       | 5       | 1           |
| 2 | 49  | 1   | 1   | 1       | 1               | 0                | 2096          | 12       | 3       | 0           |
| 3 | 45  | 1   | 2   | 0       | 1               | 1                | 7882          | 42       | 4       | 0           |
| 4 | 53  | 1   | 2   | 0       | 1               | 1                | 4870          | 24       | 1       | 1           |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Features
X = df.drop("Credit Risk", axis=1)

# Target
y = df["Credit Risk"]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
```

```
Features Shape: (954, 9)
Target Shape: (954,)
```

```
from sklearn.model_selection import train_test_split

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

Training Data: (763, 9)  
Testing Data: (191, 9)

```
from sklearn.linear_model import LogisticRegression
```

```
# Create model
```

```
lr = LogisticRegression(max_iter=1000)
```

```
# Train model
```

```
lr.fit(X_train, y_train)
```

```
# Predictions
```

```
y_pred_lr = lr.predict(X_test)
```

/usr/local/lib/python3.12/dist-packages/sklearn/linear\_model/\_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (s  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max\_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

[https://scikit-learn.org/stable/modules/linear\\_model.html#logistic-regression](https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression)

```
n_iter_i = _check_optimize_result(
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
print("Logistic Regression Results")  
print("-----")
```

```
print("Accuracy :", accuracy_score(y_test, y_pred_lr))  
print("Precision:", precision_score(y_test, y_pred_lr))  
print("Recall   :", recall_score(y_test, y_pred_lr))  
print("F1 Score :", f1_score(y_test, y_pred_lr))
```

```
Logistic Regression Results
```

```
-----  
Accuracy : 0.6963350785340314  
Precision: 0.6190476190476191  
Recall   : 0.20634920634920634  
F1 Score : 0.30952380952380953
```

```
from sklearn.metrics import roc_auc_score
```

```
y_prob_lr = lr.predict_proba(X_test)[:,:1]
```

```
roc = roc_auc_score(y_test, y_prob_lr)
```

```
print("ROC-AUC Score:", roc)
```

```
ROC-AUC Score: 0.6770833333333333
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Create Decision Tree model
```

```
dt = DecisionTreeClassifier(random_state=42)
```

```
# Train the model
```

```
dt.fit(X_train, y_train)
```

```
# Make predictions
```

```
y_pred_dt = dt.predict(X_test)
```

```
# Evaluate
```

```
print("Decision Tree Results")  
print("-----")
```

```
print("Accuracy :", accuracy_score(y_test, y_pred_dt))  
print("Precision:", precision_score(y_test, y_pred_dt))  
print("Recall   :", recall_score(y_test, y_pred_dt))  
print("F1 Score :", f1_score(y_test, y_pred_dt))  
print("ROC-AUC  :", roc_auc_score(y_test, dt.predict_proba(X_test)[:,:1]))
```

```
Decision Tree Results
```

```
-----  
Accuracy : 0.6282722513089005  
Precision: 0.42857142857142855  
Recall   : 0.38095238095238093
```

```
F1 Score : 0.40336134453781514
ROC-AUC : 0.5654761904761905
```

```
from sklearn.ensemble import RandomForestClassifier

# Create Random Forest model
rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train model
rf.fit(X_train, y_train)

# Predictions
y_pred_rf = rf.predict(X_test)

# Evaluation
print("Random Forest Results")
print("-----")
print("Accuracy :", accuracy_score(y_test, y_pred_rf))
print("Precision:", precision_score(y_test, y_pred_rf))
print("Recall   :", recall_score(y_test, y_pred_rf))
print("F1 Score :", f1_score(y_test, y_pred_rf))
print("ROC-AUC  :", roc_auc_score(y_test, rf.predict_proba(X_test)[:,:1]))
```

```
Random Forest Results
-----
Accuracy : 0.7068062827225131
Precision: 0.5897435897435898
Recall   : 0.36507936507936506
F1 Score : 0.45098039215686275
ROC-AUC  : 0.7268725198412697
```

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred_rf)

print("Confusion Matrix")
print(cm)
```

```
Confusion Matrix
[[112  16]
 [ 40  23]]
```

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_rf))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.74      | 0.88   | 0.80     | 128     |
| 1            | 0.59      | 0.37   | 0.45     | 63      |
| accuracy     |           |        | 0.71     | 191     |
| macro avg    | 0.66      | 0.62   | 0.63     | 191     |
| weighted avg | 0.69      | 0.71   | 0.68     | 191     |

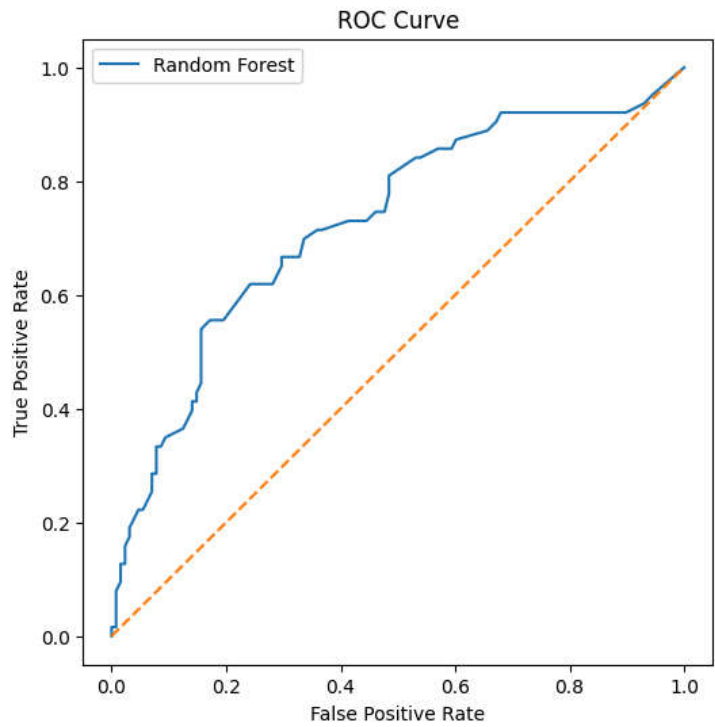
```
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

y_prob = rf.predict_proba(X_test)[:,:1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, label="Random Forest")
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```





```
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_lr),
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_rf)
    ],
    "Precision": [
        precision_score(y_test, y_pred_lr),
        precision_score(y_test, y_pred_dt),
        precision_score(y_test, y_pred_rf)
    ],
    "Recall": [
        recall_score(y_test, y_pred_lr),
        recall_score(y_test, y_pred_dt),
        recall_score(y_test, y_pred_rf)
    ],
    "F1 Score": [
        f1_score(y_test, y_pred_lr),
        f1_score(y_test, y_pred_dt),
        f1_score(y_test, y_pred_rf)
    ]
})

comparison
```

|   | Model               | Accuracy | Precision | Recall   | F1 Score |  |
|---|---------------------|----------|-----------|----------|----------|--|
| 0 | Logistic Regression | 0.696335 | 0.619048  | 0.206349 | 0.309524 |  |
| 1 | Decision Tree       | 0.628272 | 0.428571  | 0.380952 | 0.403361 |  |
| 2 | Random Forest       | 0.706806 | 0.589744  | 0.365079 | 0.450980 |  |

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

Start coding or [generate](#) with AI.

